

BALANCED ACADEMIC CURRICULUM PROBLEM

Equilibrar la malla curricular

Distribuir las asignaturas de una carrera entre semestres para que la carga académica quede lo más balanceada posible — resuelto mediante metaheurística.

TÉCNICA ASIGNADA

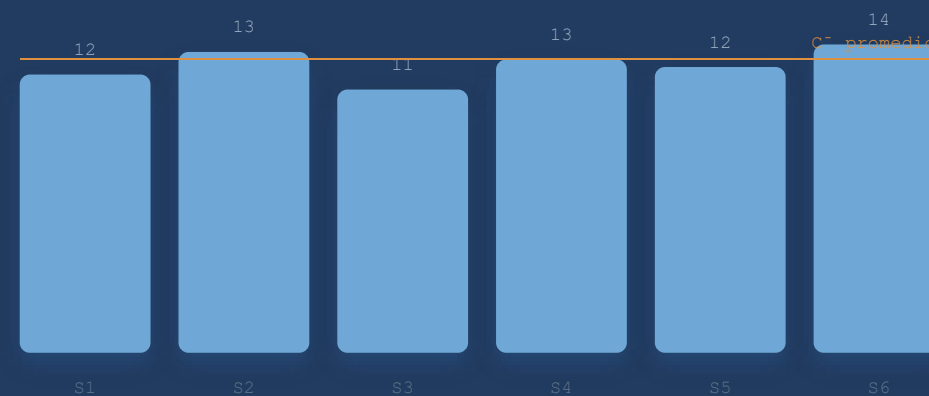
Búsqueda Tabú

ACRÓNIMO

BACP

FIG. 0 — CARGA POR SEMESTRE

CRÉDITOS



minimizar varianza → balance perfecto = 0

Definición del Problema

BALANCED ACADEMIC CURRICULUM PROBLEM

Asignar un conjunto de asignaturas universitarias a **semestres** de modo que la carga académica quede lo más **equilibrada** posible, respetando las relaciones de **prerrequisito** y las restricciones de **capacidad** semestral — mínimo y máximo de créditos y de asignaturas por semestre.

«¿Cómo distribuimos los ramos en semestres de forma que ninguno quede ni muy sobrecargado, ni muy liviano?»

FIG. 1 — CADENA DE PRERREQUISITOS

INF-134

Estructuras de Datos



INF-292

Optimización



INF-295

Inteligencia Artificial

un prereq fija el orden parcial entre semestres

Representación

FIG. 2 – INSTANCIA DE EJEMPLO

I	ASIGNATURA	CRÉD.	PREREQ
0	Cálculo I	5	—
1	Álgebra	4	—
2	Cálculo II	5	0
3	Física	4	0
4	Programación	3	—
5	Estructuras	4	4

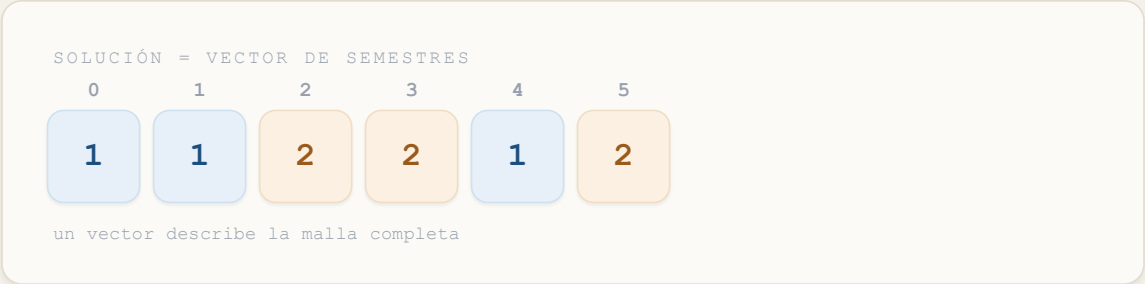


FIG. 3 – CARGA RESULTANTE

SEM.	ASIGNATURAS	CARGA
1	Cálculo I, Álgebra, Prog.	12
2	Cálculo II, Física, Estruct.	13
3	—	0

prereq ✓ sem[0]=1 < sem[2]=2

movimiento tabú = cambiar 1 valor · O(1)

Lectura de Instancia

```
// formato .dat de entrada p = 8;  a = 10;  b = 24; c = 2;  d = 10;
courses = { dew100, fis100, ... }; credit = [ 1, 3, 5, ... ]; prereq
= {
    <fis110, fis100>,
    <mat022, mat021>, ...
};
```



8 / 8 instancias cargadas correctamente
benchmark BACP + casos reales UTFSM

FIG. 4 – INSTANCIAS PROBADAS

INSTANCIA	SEM.	CURSOS	PREREQS	
bacp8	8	46	38	
bacp10	10	42	34	
bacp12	12	66	65	
utfsm3	4	23	16	
utfsm2311	12	67	93	
utfsm2920	8	46	29	
utfsm7310	10	57	43	
utfsm7313	11	59	51	

[4] Chiarandini et al., *Journal of Heuristics*, 2012 – barra = n° de prerequisitos

Solución Inicial — Greedy Topológico

1

Calcular nivel mínimo

BFS sobre el grafo de prereqs. $nivel[i] = \max(nivel[pre]) + 1$. Sin prereqs $\rightarrow$ nivel 1.

2

Ordenar asignaturas

Por nivel ascendente. Desempate: más créditos primero, para llenar los semestres con holgura.

3

Asignar greedily

Primer semestre $\geq$ nivel mínimo que no viole $carga_max$ ni $assign_max$.

!

La solución inicial **puede ser infactible** — la Búsqueda Tabú se encarga de mejorarla y repararla desde ahí, escapando de óptimos locales.

Función de Evaluación

$$f(x) = \sum (C_j - \bar{C})^2 + \sum \text{penalizaciones}$$

TÉRMINO 1 · VARIANZA DE CARGA

$$\sum (C_j - \bar{C})^2$$

- C_j = suma de créditos del semestre j
- $\bar{C}$ = carga promedio entre semestres
- valor 0 → balance perfecto

TÉRMINO 2 · PENALIZACIONES ($\times 1000$)

- carga fuera de $[C_{\min}, C_{\max}]$
- cantidad fuera de $[n_{\min}, n_{\max}]$
- prereq en semestre $\geq$ sucesor
- peso alto → prioriza factibilidad

Objetivo: minimizar $f(x)$ — una solución factible siempre es mejor que cualquier infactible.

Movimiento: Traslado (*Move*)

¿QUÉ HACE?

Reasigna la asignatura a_i desde su semestre actual s_i a uno diferente s_j :

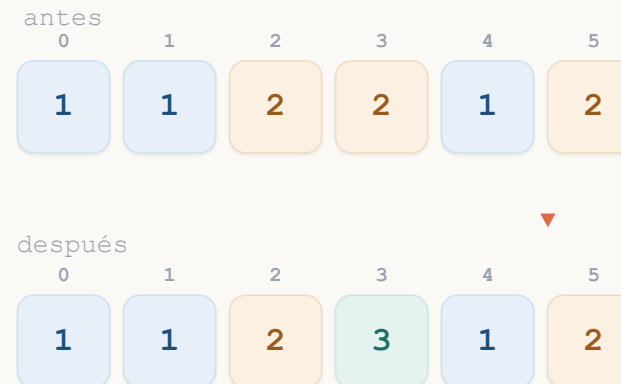
```
semestre[i] :  $s_i \rightarrow s_j$ 
```

Antes de aceptar, verifica que ningún prerrequisito de a_i quede en un semestre $\geq s_j$ (restricción dura).

¿POR QUÉ ESTE MOVIMIENTO?

Movimiento elemental base de la búsqueda local para BACP. Explora localmente a bajo costo; un cambio en `semestre[i]` modifica la carga de **dos** semestres a la vez.

FIG. 5 – MOVER «FÍSICA» ($I=3$) · $S=2 \rightarrow S=3$



Manejo de Restricciones



Restricciones Duras · Hard

- **Prerrequisitos:** solo se evalúan movimientos donde el prereq queda en un semestre estrictamente anterior al sucesor.
- Los movimientos que violan un prereq se **descartan antes** de evaluarse (poda).
- Base teórica: algoritmos híbridos CSP [8].



Restricciones Blandas · Soft

Carga y cantidad por semestre, incorporadas al fitness con penalización fija (peso $P = 1000$):

$$\text{penalización} = P \cdot (\text{magnitud de la violación})$$

- Proporcional a *cuánto* se viola -> da gradiente que guía hacia la región factible.
- $P = 1000$ es mucho mayor que el desbalance máximo factible, por lo que toda solución factible es mejor que cualquier infactible.
- Permite atravesar regiones infactibles para escapar de óptimos locales, sin reparación explícita.

Diseño de la Búsqueda Tabú

Búsqueda local que parte de una solución y la mejora iterativamente, recordando los movimientos recientes en una *lista tabú* para no deshacerlos y así escapar de óptimos locales.



Trabajo por Realizar

≈ 40% avance



Implementado

- Lectura de instancias (8/8 ✓)
- Función de evaluación con penalizaciones
- Solución inicial greedy topológica
- Verificación de factibilidad



Por implementar

- Lista tabú (longitud aleatoria)
- Movimiento Move + criterio de aspiración
- Ciclo principal de Búsqueda Tabú
- Experimentos sobre las 8 instancias
- Comparación con resultados de literatura

Conclusiones

01

Mínimo implementado

Lectura, función de evaluación y solución inicial funcionan correctamente sobre las 8 instancias benchmark y reales UTFSM.

02

Diseño

El movimiento Move está diseñado y justificado desde la literatura. Prereqs como restricción dura; capacidades con penalización fija.

03

Próximo paso

Implementar el ciclo principal de Búsqueda Tabú con lista tabú dinámica, criterio de aspiración para escapar de óptimos locales.